Comet Foundry

음승재 · Unity 게임 프로그래머 포트폴리오

게임 플레이 시스템 개발 포트폴리오



Unity 6000.3.19f1 / C#
PC 2D 산업 경영 시뮬레이션

소스 검토 기반 예시 · 기여 범위와 실행 증빙은 제출 전 확정

프로젝트 주요 시스템 소개

코멧 파운드리의 핵심 시스템을 기능별로 분리하여 데이터 생성, 관리 및 생산과정의 결과반영하고자 한다

플레이어 데이터

- 보유혜성 배열
- 자금 관리

- 거래소 신용도
- 신용도에 따른 보상

혜성 매니저

- 혜성객체의 생성 및 삭제
- 건설된 건물객체 정보
- 혜성객체의 보유 아이템
- 혜성객체의 아이템 이동
- 혜성의 좌표시스템

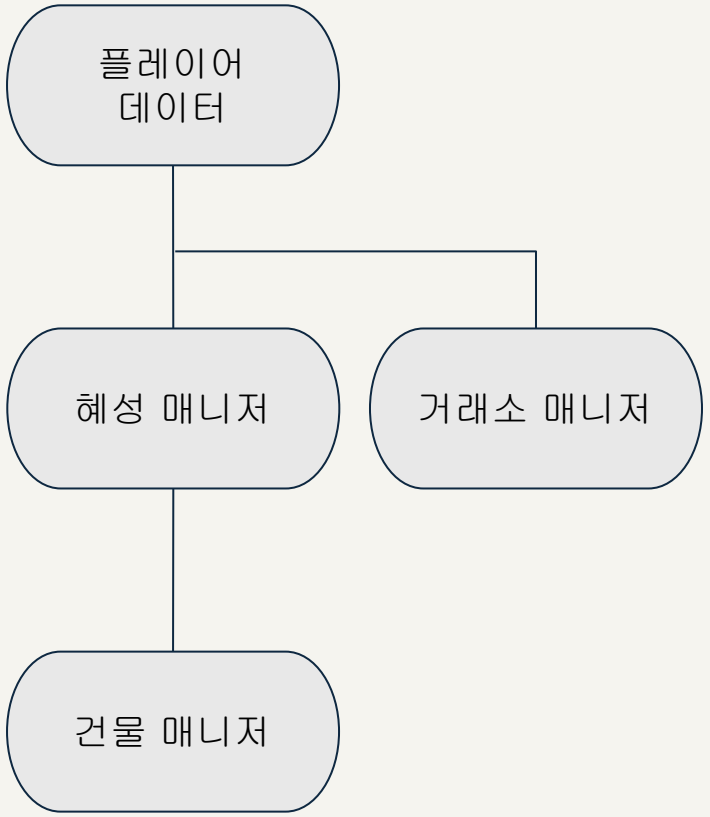
거래소 매니저

- 아이템의 매입과 매각
- 아이템의 시세 추가
- 퀘스트 및 보상

- 거래소 시세변동 차트

건물 매니저

- 건물객체의 생성 및 삭제
- 건물 역할부여
- 건물 작동 조건
- 아이템 생산
- 건물의 병렬작동



프로젝트 주요 시스템 소개

구현 목표

혜성 매니저

- **혜성의 다중생성 및 지정객체 저장 및 삭제**

혜성의 객체를 다중생성하여 해당 객체의 정보를 반영하여 유저가 원하는 객체를 선택하고 소유 및 보유객체의 삭제 기능을 구현한다.

- **건설된 건물객체 정보 관리**

혜성객체에 건설된 건물의 좌표정보, 건물의 타입정보를 저장한다.

- **혜성객체의 보유아이템 (저장소 시스템)**

혜성객체간 보유 저장소의 크기만큼 아이템 배열에 아이템을 저장한다.

- **혜성객체간 아이템 이동 (운반선 시스템)**

혜성객체간 보유한 아이템의 이동을 구현한다.

- **혜성 좌표 시스템**

혜성간 좌표를 만들고, 생성시 보유한 혜성과 다중생성된 혜성중 좌표가 중복되는지 검사후 중복시 다시생성한다.
혜성간의 좌표를 토대로 거리를 만들고 거리를 바탕으로 아이템운반시 이동시간을 부여한다.

프로젝트 주요 시스템 소개

구현 목표

건설 매니저

- **건물객체의 생성 및 삭제**

건물의 객체를 생성시 지정된 건물의 타입에 따라 크기를 책정하고 혜성의 내부 좌표값에 반영하여 건물객체를 생성한다. 또한 각 혜성에서 지정된 건물의 삭제하는 시스템이다.

- **건물 역할부여**

객체의 생성시 지정된 타입을 확인후 역할을 부여하는 시스템이다.
예) 채굴기 = 생산, 조립기 = 소비 및 생산, 발전기 = 코스트확보, 창고 = 최대 데이터 보유수

- **건물 작동 조건**

건물 객체생성의 남용을 막기위한 조치이다. 발전기의 코스트를 작동키로 하여금 건물의 작업을 진행하는 시스템이다..

- **아이템 생산**

해당 혜성의 객체에 아이템을 생산 또는 소비 및 생산의 작업을 하는 시스템이다..

- **건물의 병렬작동**

비 출력시 또는 객체마다 작업이 진행되도록 하기위한 시스템이다.

구현 범위와 개인 기여

혜성 매니저

혜성의 다중생성 및 지정객체 저장 및 삭제

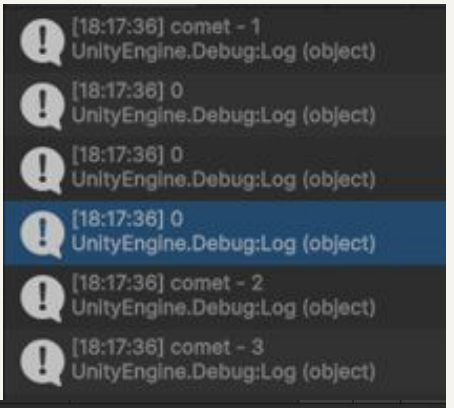
혜성의 다중생성 테스트 1차 실패 - 사유 for 반복문의 조건 결함
랜덤으로 작성된 등급에서 자원노드배열에 switch 문을 사용하여 노드의 타입을 부여하고
해당타입을 바탕으로 자원을 랜덤 부여하는 식으로 혜성의 고유자원을 부여한다.
그중 등급의 지정후 노드의 타입을 자원노드 배열에 부여하는 코드의 반복문인 코드에서

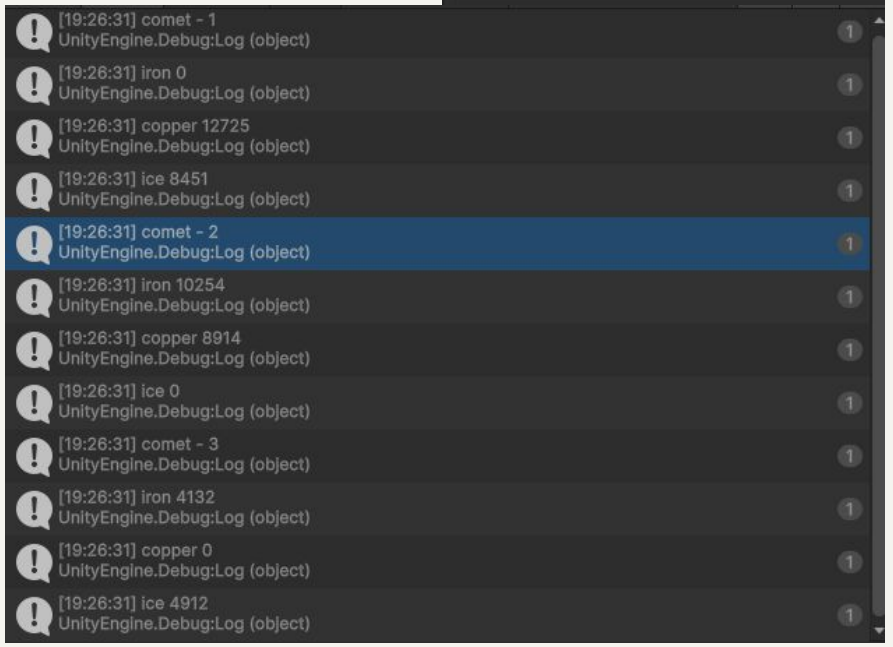
```
for (int y = 0; y > comet.resourceNodeCount; y++)
```

y값이 객체의 자원노드보유수 의 값보다 작기때문에 조건이 바로 만족되 풀려버린다.

```
for (int y = 0; y < comet.resourceNodeCount; y++)
```

으로 수정후 정상작동 확인.





테스트날짜 - 2026-10-17

구현 범위와 개인 기여

혜성 매니저

혜성의 다중생성 및 지정객체 저장 및 삭제

실제 혜성객체 자원부여 코드첨부

```
for (int y = 0; y < comet.resourceNodeCount; y++)
{     int Nodetype = Random.Range(1, 4);
      comet.totalResourceAmount[y] = Nodetype;
      // 1 = iron, 2 = copper, 3 = ice 각 배열의 번호에 맞는 자원을 랜덤으로 추가한다.
      switch (comet.totalResourceAmount[y])
      {
            case 0: break;
            case 1: comet.ironAmount = comet.ironAmount + Random.Range(2500, 5000); break;
            case 2: comet.copperAmount = comet.copperAmount + Random.Range(2500, 5000); break;
            case 3: comet.iceAmount = comet.iceAmount + Random.Range(2500, 5000); break;
      }
}
```

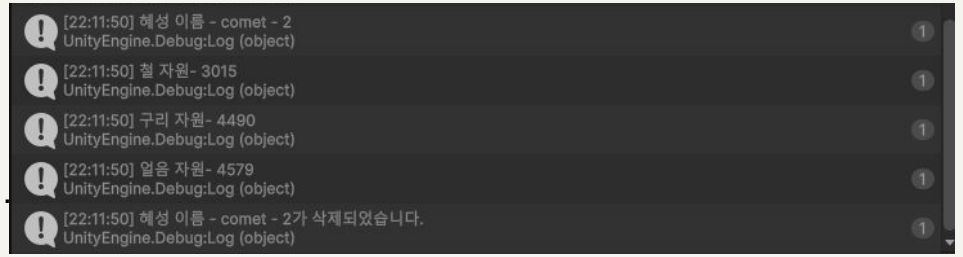
혜성의 등급에 따른 자원노드의 최대수를 부여후 배열마다 노드의 타입을 넣어 스위치 조건문으로 해당 노드의 타입에 과 일치하는 자원을 혜성객체에 부여한다.

구현 범위와 개인 기여

혜성 매니저

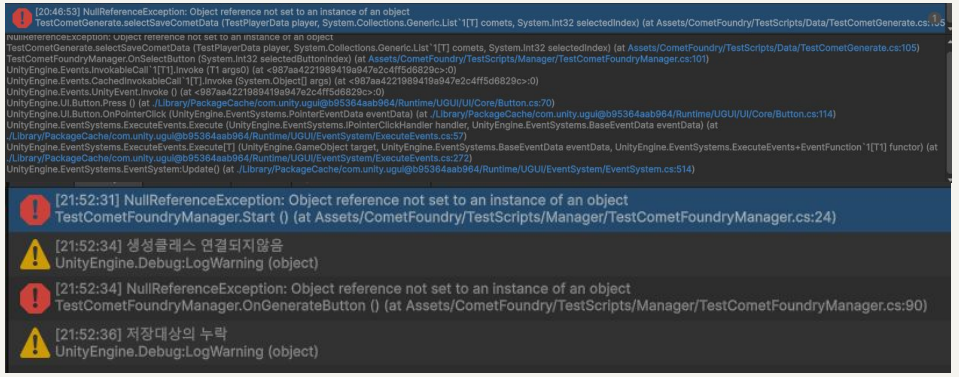
혜성의 다중생성 및 지정객체 저장 및 삭제

테스트 1차 – 사유 : 함수의 미숙지
Remove 와 RemeveAt 함수의 차이를 미숙지하여 실패하고 개선하여 성공하였다.



테스트날짜 : 2026-10-18

지정객체 저장 테스트 1차 실패 – 사유 : PlayerData와의 연결누락
자세한 사항을 찾기위해 디버그로그를 추가하여 확인을 실시하였다.
플레이어 데이터의 객체가 생성되지 않은것을 확인하으며 해결책을 알수없어
AI를 통한 피드백을 받는다.



void Start()
{ ----- TestPlayerData testPlayer = new TestPlayerData(); ----- }
해당 코드가 문제임을 지적.

Start() 메서드 내부에서 클래스 필드와 동일한 이름의 지역변수를 선언하여 변수 가리기(Variable Shadowing)가 발생하였다. 이로 인해 클래스 필드가 초기화되지 않아 NullReferenceException 이 발생하였다. 지역변수 선언을 제거하고 클래스 필드에 직접 객체를 할당하여 문제를 해결한다.

구현 범위와 개인 기여

혜성 매니저

혜성의 다중생성 및 지정객체 저장 및 삭제

지정객체 저장 수정과 결과
TestPlayerData testPlayer = new TestPlayerData();
에서 클래스타입을 지우고
testPlayer = new TestPlayerData();
만 남기고,

PlayerData 클래스에서
public List<TestCometData> userComet = new List<TestCometData>()
= new List<TestCometData>() 를 추가하였다.

해당수정은 AI를 통하여 알게된 수정안이다.

테스트 날짜 : 2026-10-18

